

AI Automation with n8n

Lesson 2: Setting Up Your Environment

Setting Up Your Environment – Study Notes

Key Concepts

1. **Local vs. Docker installation of n8n** – understanding when to use a native install versus a containerized setup.
2. **Docker basics for n8n** – pulling the image, configuring environment variables, and managing containers.
3. **API key management** – securely storing and referencing AI service credentials (OpenAI, Anthropic, etc.) inside n8n.
4. **Environment variables & `.env` files** – the preferred method for passing secrets to n8n without hard coding them.
5. **Testing the setup** – confirming that n8n is reachable and that AI credentials are recognized by workflow nodes.

Summary

This lesson walks you through getting the n8n workflow automation tool up and running on your own machine. You can install n8n directly on the host operating system (Node.js + npm) or, more commonly for beginners, run it inside a Docker container. Docker isolates n8n from the rest of your system, simplifies dependency management, and makes it easy to share the same environment across machines.

After n8n is running, the next crucial step is to configure API keys for the AI services you plan to use (e.g., OpenAI's GPT-4, Anthropic's Claude). n8n stores these keys as **environment variables**, which you can define in a `.env` file or pass directly to the Docker command. By keeping secrets out of workflow definitions, you maintain security and make your automations portable.

Finally, you'll verify the installation by opening the n8n UI, creating a simple workflow that calls an AI node, and confirming that the response is successful. This ensures both the platform and the external API credentials are correctly wired together.

Important Points

- **Docker install (recommended):**

```
```bash
docker run -d \
 --name n8n \
 -p 5678:5678 \
 -v ~/.n8n:/home/node/.n8n \
 -e N8N_BASIC_AUTH_ACTIVE=true \
```

```
-e N8N_BASIC_AUTH_USER=admin \
-e N8N_BASIC_AUTH_PASSWORD=strongpassword \
-e OPENAI_API_KEY=sk-... \
n8nio/n8n
```
```

- **Native install (Node.js):** ``npm install n8n -g`` then run ``n8n start``.
- **Use a `.env`` file** (placed in the same folder you run Docker from) to avoid exposing keys in the command line:

```
```.env
N8N_BASIC_AUTH_ACTIVE=true
N8N_BASIC_AUTH_USER=admin
N8N_BASIC_AUTH_PASSWORD=strongpassword
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
```
```

- **Never commit API keys** to version control. Add `.env`` to `.gitignore``.
- **Port mapping:** n8n's default UI runs on **5678**; you can change it with ``-p <hostPort>:5678``.
- **Persistent data:** Mount a host folder (``-v ~/.n8n:/home/node/.n8n``) so workflows survive container restarts.
- **Testing:** Open ``http://localhost:5678`` !' log in !' add an **OpenAI** node !' set **Model** !' run. If you receive a response, the environment is correctly configured.

Examples

Example 1 – Starting n8n with Docker and a `.env`` file

1. Create a project folder: ``mkdir n8n_env && cd n8n_env``.
2. Inside, create a `.env`` file:

```
```.env
N8N_BASIC_AUTH_ACTIVE=true
N8N_BASIC_AUTH_USER=admin
N8N_BASIC_AUTH_PASSWORD=My$ecurePass
OPENAI_API_KEY=sk-XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```
```

3. Run the container:

```
```.bash
docker run -d \
 --name n8n \
 -p 5678:5678 \
 -v $(pwd)/.n8n:/home/node/.n8n \
 --env-file .env \
 n8nio/n8n
```
```

4. Open `http://localhost:5678`, log in with `admin / My$ecurePass`, and you're ready to build workflows.

Example 2 – Simple “Hello AI” workflow (native install)

```
```bash
npm install n8n -g
n8n start
```
```

1. In the UI, add an **OpenAI** node.
2. Set **Model** to `gpt-4o-mini`.
3. In **Prompt**, type: `Say hello and tell me the current date.`
4. Connect the node to a **Set** node to capture the output, then click **Execute Node**.

If the node returns a JSON with a `text` field containing the greeting, your API key is correctly recognized.

Quick Review

1. **What are two main ways to install n8n, and which is generally recommended for beginners?**
2. **How should you store API keys for AI services when running n8n in Docker?**
3. **Why is mounting a host volume (`-v`) important for a Docker based n8n instance?**
4. **What default port does n8n's web UI listen on, and how can you change it?**
5. **Name one security practice you must follow when handling API keys in n8n.**

Further Reading

- **Docker Compose for n8n** – orchestrate n8n with other services (PostgreSQL, Redis) in a single `docker-compose.yml`.
- **n8n Credential Types** – deep dive into built in credential storage, encryption, and OAuth flows.
- **Managing Secrets with Docker Secrets or HashiCorp Vault** – more advanced secret management strategies.
- **Scaling n8n** – using external databases, load balancers, and Kubernetes.
- **Exploring AI nodes** – OpenAI, Anthropic, Cohere, and how to chain them in complex workflows.

